

Intel Many Integrated Core (MIC) Co-processor : a performance and programmability study

Mahesha S and Naga Vydyanathan
Siemens Corporate Research & Technologies
Bangalore - 560100, India

{mahesha.s, nagavijayalakshmi.vydyanathan}@siemens.com

Rama Kishan Malladi V and Vipin Kumar E K
Intel Technology India Pvt. Ltd
Bangalore - 560017, India

{rama.kishan.v.malladi, vipin.kumar.e.k}@intel.com

Abstract—Heterogeneous computer architectures, where CPUs co-exist with accelerators such as vector co-processors, GPUs and FPGAs, are rapidly evolving to be powerful platforms for tomorrow's exa-scale computing. Intel is one of the key players in many-core and accelerator architectures of the future. The Intel Many Integrated Core or Intel MIC (pronounced Mike) project is Intel's first step towards heterogeneous computing. The MIC prototype product, codenamed Knights Ferry (KNF) was announced and released in late 2010, and a commercial release, codenamed Knights Corner (KNC), to be built on a 22nm process, is proposed for release in late 2012 or early 2013. This paper explores the performance and programmability aspects of the Knights Ferry platform in the context of medical image processing. Using a well known and widely used signal and image processing algorithm - complex finite impulse response (FIR) filtering, we evaluate the ease of programming on the MIC platform as well as its performance efficiency. Our analysis indicates that the Intel MIC is an 'easy-to-program' accelerator of the future, that also shows good potential in terms of scalability and performance.

Index Terms—parallel computing, multi-core, many-core, accelerators, heterogeneous programming.

I. INTRODUCTION

Growing complexities, scale and performance needs of applications across all domains, be it scientific computing or business, has placed heavy demands for powerful computational resources. Though technology development has resulted in the ability to add more and more transistors on a single chip, the CPU clock speeds have hit a plateau due to several factors - heat and power barriers, memory wall, propagation delays and other physical limits. This, coupled with the limits on ILP (instruction level parallelism) due to dependences as well as hardware limitations (register renaming, branch mispredictions, instruction window sizes etc.), has led to the rise of multi-core systems and thread level parallelism (TLP) [1].

However, multi-core systems alone cannot scale up to the computing needs of the future. Today's multi-core CPUs spend a lot of transistors and power on non-computational units such as logic and cache, and hence the number of cores grows sub-linearly with the number of transistors. On the other hand, accelerators such as the GPUs devote more transistors to data processing rather than cache and control, and hence, increase cores almost linearly with transistor counts. These accelerator cores are not suited for general purpose computing and are managed by the CPU which offloads heavy computations to

them. They are larger in number, operate at lower frequencies and power and use lesser number of transistors. As a result, accelerators provide more computing cores for lesser price and lesser power in comparison to multi-cores. Thus, heterogeneous architectures that combine general purpose multi-core CPUs with lightweight special purpose accelerators provide powerful computing platforms with attractive performance-to-price and most importantly performance-to-power ratios, and are crucial players in the road to exascale and green computing [2], [3].

However, one of the major pitfalls of accelerators like GPUs is their lack of easy programmability. General purpose software needs to be re-written in specific low level languages like NVIDIA CUDA [4], OpenCL [5] or Microsoft DirectCompute [6] to execute on GPUs. As a first step towards striking a balance between programmability and performance, Intel has initiated an x86 based co-processor architecture called Many Integrated Core (MIC). The MIC project is a culmination of three research projects at Intel: the 80 core tera-scale research chip, the single chip cloud computer initiative and the Larrabee GPGPU chip. The MIC architecture combines a large number of smaller x86 cores with lower single threaded performance and operating at lower power, to deliver a high aggregate performance for highly parallel workloads. Knights Ferry is a design and development kit based on the MIC architecture released in late 2010 and Knights Corner is the first commercial MIC product to be marketed in late 2012 or early 2013.

This paper evaluates the Intel MIC co-processor, specifically, the Knights Ferry (KNF) platform, from a performance and programmability perspective. Using a well known and widely used signal and image processing algorithm - complex finite impulse response (FIR) filtering, we investigate the ease of programming on the MIC platform as well as its performance efficiency. We study the effort needed to port and optimize the FIR filter algorithm on KNF and evaluate its scalability characteristics. Our analysis indicates that the Intel MIC is an 'easy-to-program' accelerator of the future, that also shows good potential in terms of scalability and performance.

The rest of this paper is organized as follows. Section II gives an overview of the architecture of the Intel MIC co-processor and the Knights Ferry Platform. Section III describes programming models and languages that can be used

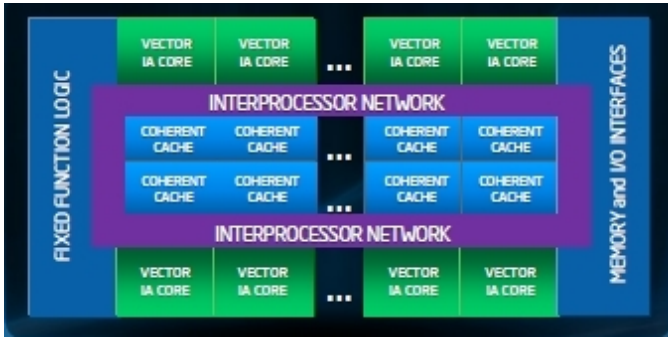


Fig. 1. The Intel MIC Architecture.

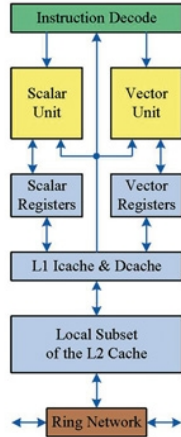


Fig. 2. x86 based MIC Core and associated System Blocks. (Courtesy: <http://inside.hlrs.de> [7])

to program the MIC co-processor. Section IV presents our evaluations of the MIC co-processor using the FIR filter algorithm. Section V captures the highlights of our experiences and observations and Section VI concludes the paper.

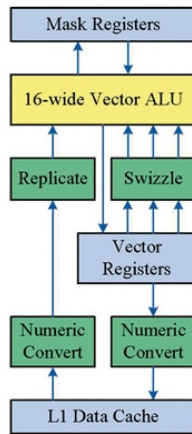


Fig. 3. Block Diagram of MIC Vector Processing Unit (VPU). (Courtesy: <http://inside.hlrs.de> [7])

II. INTEL MIC ARCHITECTURE

Figure 1 shows a schematic illustration of the Intel MIC architecture. The MIC co-processor consists of a set of x86 cores (Figure 2), each augmented with a wide vector processing unit (VPU) (Figure 3). Each core runs an extended version of the x86 instruction set that includes wide vector processing operations. Every core has its low latency private L1 cache and the L2 cache is spread across the cores with each core having a local subset. The cores have a fast direct access to their subset of the L2 cache and coherency is maintained across the subsets. A bi-directional high bandwidth ring network is used to communicate between the cores, L2 caches and other logic blocks.

The Knights Ferry kit, which is the first prototype implementation of the MIC architecture has 32 x86 cores, each with a 32KB L1 instruction cache and a 32KB L1 data cache and a 256KB local subset of the L2 cache. Each core has a 512 bit wide vector unit that supports up to 16 single precision floating point operations within a single instruction. Double precision compute throughput is one-eighth that of single precision. The L2 caches are kept coherent through a high bandwidth ring interconnect. Each core supports up to four threads.

The Knights Ferry is implemented as a co-processor on a PCIe card. It has its own memory and is connected to the host memory through PCI DMA operations. The Knights Corner, which is the first commercial MIC product to be released in the market in late 2012 or early 2013, is expected to have more than 50 cores and manufactured using Intels 22nm process and trigate transistors.

An important highlight of the MIC architecture is the addition of new VPU instructions, which allows a variety of operations on both integer and floating point data types, such as standard arithmetic operations including fused multiply-add, standard logical operations, and gather and scatter operations. The VPU instructions allow up to three source operands, one of which can directly come from the L1 cache. 8-bit and 16-bit integer and floating point numbers can be converted to 32-bit integer and floating point numbers without loss of performance. The VPU supports swizzling of register data in a variety of ways and replication of data from memory across the VPU lanes [7].

In summary, following are the architectural features of MIC.

- The MIC co-processor cores are based on the x86 architecture. They are simpler, operate at lower frequencies, but are more in number than traditional multi-cores and allow for high compute throughput
- The expensive x86 processor features such as out-of-order execution, branch prediction, etc. are stripped out in MIC.
- The MIC device supports fast (GDDR5) memory on card
- Much of the silicon on MIC is dedicated to floating point operations while keeping some cache(s)
- The cache-coherency protocol is maintained
- MIC is implemented as a separate PCI express device

The next section describes the programming models supported on MIC.

III. INTEL MIC PROGRAMMING MODEL

Heterogeneous accelerator-based programming is cumbersome and effort-intensive as compared to traditional CPU programming due to two main reasons: a) the instruction sets on the host CPU and the accelerators may not be identical, and b) the host and the accelerators do not share common system memory. The program execution typically begins on the host CPU and some user-defined sections of the code and the corresponding data to be processed, are offloaded by the CPU to the accelerators. However, recently, key players in accelerator-based computing such as NVIDIA and Intel are looking at solutions to mitigate the lack of easy programmability of these systems. Some innovations are the support for unified address space between the host and NVIDIA GPUs [8] and the launch of x86 based co-processor architectures such as the Intel MIC.

As the Intel MIC co-processor is based on x86, existing standard parallel programming models like OpenMP can be easily extended to MIC. This makes the task of porting legacy applications to MIC much easier as compared to GPGPU programming which requires re-writing of applications using low-level languages such as CUDA and OpenCL. As of today, MIC supports two programming models through language extensions: OpenMP and Intel Cilk Plus. This paper only discusses about the OpenMP extended MIC programming model.

The OpenMP extended MIC programming model uses a OpenMP like pragma for C/C++, or a OpenMP like directive for FORTRAN, to mark the section of the code that is to be offloaded to run on the MIC co-processor(s). The statement or the block of code following the “offload” pragma or directive is converted into an outlined function that runs on MIC. An example is shown in Figure 4 where the code section that performs the vector addition is offloaded to MIC by using the pragma “#pragma offload target(mic)”. Also, note that the code is parallelized by using OpenMP “parallel for” construct to take advantage of the multiple co-processor cores.

The functions and global variables to be compiled for the MIC platform have to be prefixed with a keyword “__declspec(target(mic))” or “__attribute__((target(mic)))”. All functions in the program are always compiled for the CPU and are available to be called on the CPU. However, only functions marked with the above keyword are available to be called by MIC-offloaded code. Global variables are treated in a similar fashion. All global variables are always present in the CPU code. But only global variables with the target attribute are compiled into the code offloaded to the co-processor. Compiling only functions and data explicitly marked with the target attribute into the Intel MIC architecture binary ensures that the code on the co-processor is as small as possible. For example, in Figure 5, the global variable “global” and the function “foo” are compiled for MIC by prefixing them with the keyword “__declspec(target(mic))”. However, as mentioned above, both the global variable “global” and the function “foo” are also compiled for the CPU and are available

```
/* *****  
 * A simple vector addition program to illustrate *  
 * MIC programming model                        *  
 * ***** */  
int main() {  
    float  a[1024]; /* first input vector */  
    float  b[1024]; /* second input vector */  
    float  c[1024]; /* output vector */  
  
    /* initialize the input vectors */  
    for(int i=0; i<1024; i++) {  
        a[i] = b[i] = i;  
    } //end for i  
  
    /* 'offload' pragma which informs the  
     * compiler to offload the code section  
     * which follows the 'offload' pragma  
     */  
    #pragma offload target(mic)  
    /* openmp 'parallel for' construct is inserted  
     * here to take advantage of MIC cores  
     */  
    #pragma omp parallel for default (shared)  
    /* vector addition operation */  
    for(int i=0; i<1024; i++) {  
        c[i] = a[i] + b[i];  
    } //end for i  
} //end main  
/* ***** */
```

Fig. 4. A simple MIC program illustration.

to be called on the CPU.

The variables whose data has to be copied to/from the co-processor have to be indicated as strictly an input or strictly an output or an input-output variable to the target region by using the offload clauses “in”, “out” and “inout” respectively. For example, in the example in Figure 5, the local variable “i” is passed as strictly an output variable and the global variable “global” is passed as an inout variable.

The functions and data referenced within offloaded code that do not have the target attribute are compiled only for the CPU and are available to be called only on the CPU. The compiler issues warnings in these scenarios. For example, in Figure 6, the global variable “global” and the function “foo” are not prefixed with the target attribute and hence are compiled only for CPU. In this case, compiler generates a warning, ignores the offload call and executes the code on the CPU.

Memory management on the CPU for pointer variables used in offloaded code is the same as for non-offloaded programs. That is, the offload pragmas do not affect memory allocation and freeing on the CPU. It must be done by the programmer as usual. However, all the necessary memory management on the MIC co-processor for pointer variables is done automatically by the compiler and runtime system. The user only needs to mention all the variables (which are being passed to the co-processor) along with the “offload” pragma or directive. There are also clauses like “alloc_if” and “free_if” which are supported for one time memory allocation and re-use of the same memory for the repeated offload calls. Figure 7 explains the usage of “alloc_if” and “free_if” clauses.

Since MIC is based on the x86 architecture, existing stan-

```

/*****
 * A MIC program example which shows the function *
 * and global variable compiled for MIC.          *
 *****/

/* the attribute __declspec( target (mic) ) below
 * informs the compiler that the global variable
 * 'global' has be compiled for MIC */
__declspec( target (mic) ) int global = 55;

/* the attribute __declspec( target (mic) ) below
 * informs the compiler that the function 'foo'
 * has be compiled for MIC */
__declspec( target (mic) ) int foo() {
    return ++global;
} //foo

int main() {
    int i;
    /* local variable 'i' is passed to MIC as output
     * where as global variable 'global' is
     * passed to MIC as both input and output */
    #pragma offload target(mic) \
        inout(global) out(i)
    {
        i = foo();
    }

    printf("global = %d, i = %d (same values)\n",
        global, i);
} //main
/*****/

```

Fig. 5. The MIC offload compilation of a function and a global variable

```

/*****
 * A program example which shows the function and *
 * global variable not compiled for MIC.          *
 *****/

/* the global variable 'global' is not compiled for
 * MIC since it is not prefixed with the attribute
 * __declspec( target (mic) ) */
int global = 55;

/* the function 'foo' is not compiled for MIC
 * since it is not prefixed with the attribute
 * __declspec( target (mic) ) */
int foo() {
    return ++global;
} //foo

int main() {
    int i;
    /* the compiler ignores the MIC offload call
     * and invokes the corresponding CPU version
     * of the call */
    #pragma offload target(mic) \
        inout(global) out(i)
    {
        i = foo();
    }

    printf("global = %d, i = %d (same values)\n",
        global, i);
} //main

```

Fig. 6. A program example where MIC offload call is ignored.

standard native C/C++ compilers, Intel standard native parallel programming models like ArBB and TBB, MKL, Pthreads, MPI, etc. are also supported on MIC.

IV. COMPLEX FIR FILTER PROCESSING ON MIC

This section discusses the implementation and performance optimization of the complex FIR filter algorithm on the Knights Ferry (KNF) platform. We first give a brief overview of the FIR algorithm, followed by a detailed discussion of the optimization strategies employed and the performance evaluation on KNF.

A. FIR Filter Algorithm

Finite Impulse Response or FIR filters [9] are one of the primary types of filters used in Digital Signal Processing. An N^{th} -order filter output $y[j]$ for the current input value $x[j]$ can be expressed as the weighted sum of the current input value $x[j]$ and N previous input values as mathematically shown below,

$$y[j] = \sum_{i=0}^N c_i x[j-i] \quad (1)$$

where:

- $x[j]$ is the input signal,
- $y[j]$ is the output signal,
- c_i are the filter coefficients, also known as tap weights, that make up the impulse response,

- and N is the filter order; an N^{th} -order filter has $(N+1)$ summation terms

We have used a “symmetric complex FIR filter” for our evaluations. In the case of complex FIR filter, the values of input, output and filter coefficients contain both real and imaginary components.

Figure 8 shows the code snippet for applying an N^{th} -order symmetric complex FIR filter on a 2D image of size $ySize \times xSize$. Note that the real and imaginary values of the pixels of the input image are stored consecutively in the 2D array ‘src’, and the real and imaginary filter coefficients are stored consecutively in the 1D array ‘coeff’.

B. Performance Evaluation of FIR Filter Algorithm on MIC

We adopt an incremental approach for optimizing the FIR filter algorithm on MIC. The FIR base kernel code (shown in Figure 8) is first parallelized, then vectorized, and then some additional optimizations are implemented to minimize the kernel compute time. Our evaluations are done on the Linux platform (version 2.6.32-71.el6.x86_64), and the compiler flag ‘-O3’ is turned on. The one-time MIC offload initialization overhead, data transfer times to/from MIC and FIR kernel computation times are captured separately. We evaluate the FIR filter computation performance by varying the filter order.

1) *FIR Filter Kernel Parallelization*: The FIR kernel code is parallelized to take advantage of the MIC co-processors using the OpenMP programming model [10]. The parallelization is done at the level of the outer most ‘for’ loop so that different threads can work in parallel on different image

```

#define ALLOC    alloc_if(1)
#define FREE    free_if(1)
#define RETAIN  free_if(0)
#define REUSE   alloc_if(0)

/* The compiler allocates and frees memory
 * for p around the offload. */
#pragma offload target(mic) \
    in( p:length(1) )
...

/* The compiler allocates memory for p and
 * keeps the memory allocated for p after
 * the offload. */
#pragma offload target(mic) \
    in( p:length(1) ALLOC RETAIN)
...

/* The offload reuses the memory allocated
 * for p previously. It only transfers fresh
 * data into that memory, and after the
 * offload completes, it continues to retain
 * the memory. */
#pragma offload target(mic) \
    in( p:length(1) REUSE RETAIN)
...

/* The offload reuses the memory allocated
 * for p previously. However, it frees the
 * memory for p after the offload. */
#pragma offload target(mic) \
    in( p:length(1) REUSE FREE)
...

```

Fig. 7. Illustration of MIC alloc_if and free_if clauses.

rows (see Figure 10). The kernel is compiled for the MIC platform using the keyword “__declspec(target(mic))” and offloaded for execution using the offload pragma mentioned in Section III. Thus, the additional lines of code that need to be written to port the base kernel shown in Figure 8 to MIC is minimal.

On the other hand, Figure 9 shows the CUDA code snippet for the FIR kernel. In contrast to MIC programming, we see that porting of the FIR algorithm to GPUs using CUDA is more effort intensive. The kernel needs to be re-written with special attention to low-level thread indexing, explicit shared memory management and thread synchronization. In addition, memory allocation on the GPU and data transfers between the host and the GPU, have to be explicitly managed by the programmer. In the case of MIC, the data movement can be implicitly managed by the compiler. Thus, the MIC platform seems to be an easier-to-program accelerator.

The next section describes the techniques used to optimise the FIR kernel on MIC.

2) *FIR Filter Kernel Optimization*: We have implemented the following optimizations to the baseline kernel to minimize the compute time on MIC.

```

void ComplexFilterFIR_NoOptimization(
    float *src,
    float *dest,
    float *coeff,
    int xSize,
    int ySize,
    int N) {
    const int HN = N/2;
    for (int k=0; k<ySize; k++) {
        int kInd = k*xSize*2;
        for (int j=0; j<xSize; j++) {
            float sumI=0;
            float sumQ=0;
            float I,Q,C,D;
            for (int i=0; i<=N; i++) {
                if ((j+i-HN)>=0) && (j+i-HN<xSize)) {
                    I = src[kInd + (j+i-HN)*2];
                    Q = src[kInd + (j+i-HN)*2 + 1];
                    C = coeff[(N-i)*2];
                    D = coeff[(N-i)*2+1];
                    sumI += I*C-Q*D;
                    sumQ += Q*C+I*D;
                } //end if
            } //end for i
            dest[kInd + j*2] = sumI;
            dest[kInd + j*2 + 1] = sumQ;
        } //end for j
    } //end for k
} //end ComplexFilterFIR_NoOptimization

```

Fig. 8. FIR Filter Base Code Snippet.

```

#define LINE_OFFSET (TAPS_/2)
#define HALF_TAPS ((TAPS_-1)/2)

__global__ void Complex_FIR_Kernel(float2 *d_in, float2 *d_out,
    int xSize, unsigned int yPitchInFloat2, int coeffOffset)
{
    __shared__ float2 shMemFloat2[4096+TAPS_];
    /* low-level thread management */
    int i = blockIdx.y*yPitchInFloat2 + threadIdx.x;

    /* coalesced reads of float2 (8 bytes per thread) */
    for (int k=threadIdx.x; k<xSize; k+=blockDim.x) {
        /* explicit shared memory management */
        shMemFloat2[k+LINE_OFFSET] = d_in[i];
    } //for end

    if (threadIdx.x < (TAPS_/2)) {
        shMemFloat2[threadIdx.x].x = 0;
        shMemFloat2[threadIdx.x].y = 0;
        shMemFloat2[threadIdx.x+LINE_OFFSET+xSize].x=0;
        shMemFloat2[threadIdx.x+LINE_OFFSET+xSize].y=0;
    } //if end

    __syncthreads(); /* synchronization */

    for (int k=threadIdx.x; k<xSize; k+=blockDim.x) {
        float2 sum={0,0};
        #pragma unroll
        for (int j=0; j<TAPS_; j++) {
            float2 Z=shMemFloat2[k+LINE_OFFSET+j-HALF_TAPS];
            float C=d_fcoeff[coeffOffset+TAPS_-1-j].x;
            float D=d_fcoeff[coeffOffset+TAPS_-1-j].y;
            sum.x+=Z.x*C-Z.y*D;
            sum.y+=Z.y*C+Z.x*D;
        } //for end
        d_out[i] = sum;
    } //for end
} //Complex_FIR_Kernel end

```

Fig. 9. FIR Filter CUDA Code Snippet.

```

__declspec( target (mic) )
void ComplexFilterFIR_OptVectorization(
    float *real_src, float *img_src,
    float *real_coeff, float *img_coeff,
    float *dest, int xSize, int ySize,
    int N) {
    const int OFFSET = (N+1)/2;
    const int HN = N/2;
    #pragma omp parallel default (shared)
    {
        #pragma omp for nowait
        for (int k=0; k<ySize; k++) {
            int kInd1 = k*(xSize+N+1);
            int kInd2 = k*((xSize)*2);
            for (int j=0; j<xSize; j++) {
                float sumI=0;
                float sumQ=0;
                float I,Q,C,D;
                for (int i=0; i<=N; i++) {
                    I = real_src[kInd1+j+OFFSET+i-HN];
                    Q = img_src[kInd1+j+OFFSET+i-HN];
                    C = real_coeff[N-i];
                    D = img_coeff[N-i];
                    sumI += I*C-Q*D;
                    sumQ += Q*C+I*D;
                } //end for i
                dest[kInd2 + j*2] = sumI;
                dest[kInd2 + j*2 + 1] = sumQ;
            } //end for j
        } //end for k
    } //end parallel
} //end ComplexFilterFIR_OptVectorization

```

Fig. 10. FIR Filter Parallelized and Vectorized Code Snippet.

```

__declspec( target (mic) )
void ComplexFilterFIR_OptLoopSplitting(
    float *real_src, float *img_src,
    float *real_coeff, float *img_coeff,
    float *dest, int xSize, int ySize,
    int N) {
    const int OFFSET = (N+1)/2;
    const int HN = N/2;
    #pragma omp parallel default (shared)
    {
        #pragma omp for nowait
        for (int k=0; k<ySize; k++) {
            int kInd1 = k*(xSize+N+1);
            int kInd2 = k*((xSize)*2);
            for (iOuter=0; iOuter<=N; iOuter+=16) {
                float C[16], D[16];
                for (int l=iOuter; l<(iOuter+16); l++) {
                    C[l-iOuter] = real_coeff[N-l];
                    D[l-iOuter] = img_coeff[N-l];
                } //end for l
                for (int j=0; j<xSize; j++) {
                    float sumI=0, sumQ=0;
                    float I,Q,C,D;
                    for (int i=iOuter; i<(iOuter+16); i++) {
                        I = real_src[kInd1+j+OFFSET+i-HN];
                        Q = img_src[kInd1+j+OFFSET+i-HN];
                        sumI += I*C[i-iOuter]-Q*D[i-iOuter];
                        sumQ += Q*C[i-iOuter]+I*D[i-iOuter];
                    } //end for i
                    dest[kInd2 + j*2] += sumI;
                    dest[kInd2 + j*2 + 1] += sumQ;
                } //end for j
            } //end for iOuter
        } //end for k
    } //end parallel
} //end ComplexFilterFIR_OptLoopSplitting

```

Fig. 11. FIR Filter Code Snippet for Loop Splitting Optimization.

a) *Vectorization*: As mentioned in Section II, each MIC core is augmented with a 512-bit wide vector processor unit (VPU) that supports up to 16 single precision floating point operations within a single instruction. In order to take advantage of the MIC VPUs, we have vectorized the inner most loop of our FIR filter algorithm. For the loop vectorization to happen, the MIC compiler expects that there are no conditional branch statement(s) within the loop [11], [12], [13], [14]. So, we had to remove the ‘if’ statement present within the inner most loop of our baseline kernel (refer to Figure 8). We have addressed this by sufficiently padding each row of the input image with zero values on either sides. With this change, though the MIC compiler could vectorize the modified code, we did not get much performance benefit from the vectorization because of interleaved data accesses to the real and imaginary values. So, we stripped the real and imaginary values of both image and coefficient data to separate arrays. With this modification, we obtained an average of 36% reduction in compute time over non-vectorized code. Detailed performance analysis is presented in Section IV-B3. The vectorized code snippet is shown in Figure 10.

b) *Loop Splitting*: The vectorized FIR filter code snippet shown in Figure 10 suffers from repeated loading of coefficient values (both real and imaginary) for every image pixel. We applied an optimization called ‘loop splitting’ [13], [15] to avoid this repeated loading of coefficient values. As shown in

Figure 11, the inner most loop is split into two loops: the original loop with index ‘i’, and the new loop with index ‘iOuter’. Please note that the iteration space of the original inner most loop of Figure 10 is shared between the two split loops of Figure 11. Having done this, we could hoist the loading of 16 values of C and D to outside the ‘j’ loop. Thus the loading of C and D values become loop invariant over the two inner most loops.

Applying the loop splitting optimization resulted in about 52% reduction in compute time over the vectorized code. Further, this opened up opportunity for another important optimization, called ‘loop unrolling’ which is explained in the next section.

c) *Loop Unrolling*: Loop unrolling [13], [15] is a loop optimization technique which replicates the body of a loop some number of times called the unrolling factor ‘u’ and iterates by step ‘u’ instead of step 1. Unrolling can improve the performance by reducing the loop overheads, increasing instruction parallelism, and improving register, data cache, or TLB locality.

We implement loop unrolling in the FIR filter algorithm for higher filter orders, i.e $N \geq 127$. For these filter orders, the ‘iOuter’ loop is incremented in steps of 128 and hence the inner most ‘i’ loop trip count is increased to 128. This allows us to unroll the inner most ‘i’ loop with an unrolling factor of 8. Figure 12 shows the FIR filter code snippet for the loop

```

__declspec( target( mic) )
void ComplexFilterFIR_OptLoopUnrolling(
    float *real_src, float *img_src,
    float *real_coeff, float *img_coeff,
    float *dest, int xSize, int ySize,
    int N) {
    const int OFFSET = (N+1)/2;
    const int HN = N/2;
    #pragma omp parallel default (shared)
    {
        #pragma omp for nowait
        for (int k=0; k<ySize; k++) {
            int kInd1 = k*(xSize+N+1);
            int kInd2 = k*((xSize)*2);
            for(iOuter=0; iOuter<=N; iOuter+=128) {
                float C[128], D[128];
                for (int l=iOuter; l<(iOuter+128); l++) {
                    C[l-iOuter] = real_coeff[N-l];
                    D[l-iOuter] = img_coeff[N-l];
                } //end for l
                for (int j=0; j<xSize; j++) {
                    float sumI=0, sumQ=0;
                    float I,Q,C,D;
                    #pragma unroll(8)
                    for (int i=iOuter; i<(iOuter+128); i++) {
                        I = real_src[kInd1+j+OFFSET+i-HN];
                        Q = img_src[kInd1+j+OFFSET+i-HN];
                        sumI += I*C[i-iOuter]-Q*D[i-iOuter];
                        sumQ += Q*C[i-iOuter]+I*D[i-iOuter];
                    } //end for i
                    dest[kInd2 + j*2] += sumI;
                    dest[kInd2 + j*2 + 1] += sumQ;
                } //end for j
            } //end for iOuter
        } //end for k
    } //end parallel
} //end ComplexFilterFIR_OptLoopUnrolling

```

Fig. 12. FIR Filter Code Snippet for Loop Unrolling Optimization.

unrolling optimization. Though, for the lower filter orders, applying loop unrolling did not result in any performance improvement, for the higher filter orders ($N \geq 127$), applying this optimization resulted in an average of 24% reduction in compute time over the vectorized code with loop splitting.

3) *FIR Filter Performance Results:* Having discussed the various kernel optimizations in Section IV-B2, we present our performance evaluation results for these optimizations in this section. Though, MIC supports up to four threads per core, we could get the best performance by launching just two threads per core. Figure 13 shows the relative performance of the FIR kernel on KNF for different filter orders and optimizations for an image size of 4096×512 . The base (unoptimized) kernel is taken as the baseline of 1.0, i.e the relative performance for any optimization is computed as the ratio of the kernel compute time for that optimization to the base (unoptimized) kernel compute time. We note that vectorization yields up to 2x and an average of 1.6x performance improvement over the base kernel. Loop splitting yields an additional 2x boost over the vectorized kernel and loop unrolling yields an additional 1.4x performance improvement over loop splitting. Thus, these optimizations play an important role in efficiently utilizing the MIC cores.

With respect to MIC initialization and data transfer times, we observed a one-time initialization overhead to initiate the application binary onto MIC, of 530 milliseconds. The data transfer times (about 16 MB of data is transferred between the host memory and KNF) was noticed to be 168 ms on an

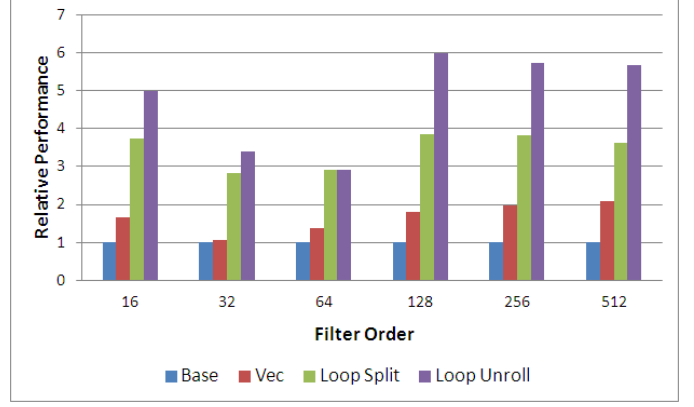


Fig. 13. FIR Filter Kernel Compute Performance on KNF. (Note: Only relative performance numbers are presented as KNF is not a product)

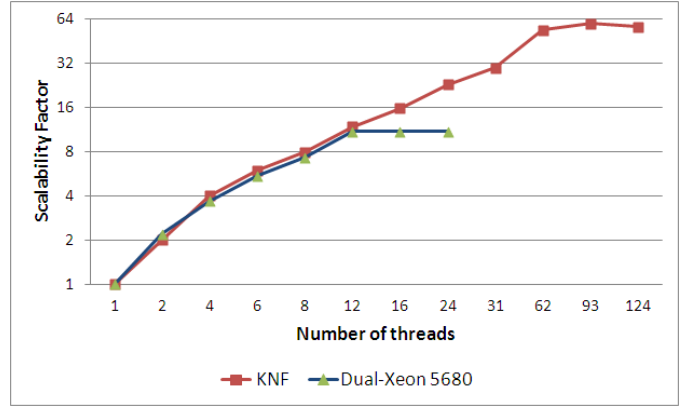


Fig. 14. FIR Filter Kernel Scalability on KNF and dual-Xeon 5680 CPU.

average with a standard deviation of 30 ms.

An interesting observation that we made was with respect to the alignment of data accesses. For best performance, the MIC VPU's expect the vector data accesses to be aligned to 64 byte boundaries. However, the access pattern of the FIR filter computation results in 4-byte aligned accesses. Therefore, for applications whose data accesses are aligned to 64 byte boundaries, we can expect a better performance on MIC.

We also compare the scalability of the FIR filter on MIC (KNF) with that on an Intel multi-core dual Xeon 5680 CPU. Each Xeon 5680 processor has 6 cores clocked at 3.33 GHz and can support two threads per core. It has a 64 KB L1 cache, 256 KB L2 cache, 12 MB L3 cache and a memory size of 24 GB. Please note that the same FIR kernel optimized for MIC is run as-is on the Xeon cores using OpenMP.

Figure 14 plots the scalability curve for the FIR kernel on KNF and dual Xeon 5680 CPU as we increase the number of threads. These experiments are done using the fully optimized FIR kernel, i.e vectorized kernel with loop splitting and loop unrolling, and an input image size of 4096×512 and filter order of 256. We see that both KNF and Xeon show close to linear scaling until 12 threads. Though Xeon supports up to 2 threads per core using hyperthreading, i.e 24 threads in all, as the FIR kernel has streaming data accesses and hence

is memory bound, we see that hyperthreading does not yield benefit and the scalability saturates with the number of cores, i.e 12. KNF, on the other hand, shows good scalability until 62 threads (i.e 2 threads per core, as KNF has 31 cores used for computations). This is because, the MIC cores are in-order and the resources on a core such as the VPU, load/store buffers etc., are not fully utilized by a single thread. Hence when two or more threads are mapped to a core, we see better performance.

Finally, to conclude, our experiments indicate that MIC as an accelerator shows good potential in terms of performance and scalability while being relatively easier to program than GPUs. As KNF is only a prototype, we expect that the commercial product KNC, which is also expected to have a larger number of cores, will show even greater performance and efficiency.

The next section summarises our experiences on the MIC platform.

V. DISCUSSION

Intel MIC is an Aubrey Isle [16] based platform. It is an x86-based many-core processor that has less powerful in-order cores that combines the full programmability of general-purpose CPU architectures with the compute throughput and memory bandwidth capabilities of modern GPU architectures. Each core is general-purpose, and has a scalar unit based on the Pentium processor design, as well as a vector unit that supports up to sixteen 32-bit float or integer operations per clock.

As MIC is based on x86, it provides a natural extension to conventional x86 programming models, like OpenMP. We found that MIC programming has an easy learning curve. With a knowledge of OpenMP, basic porting of a program to execute on MIC can be achieved very quickly. Further, the MIC programming model abstracts the programmer from the job of handling device memory. Device memory allocation/de-allocation, data transfer to/from device memory, etc. is implicitly addressed by the MIC compiler. At the same time, clauses like the 'alloc_if', 'free_if' and 'no_copy' are provided to allow the programmer to specify hints to the MIC compiler to optimize data transfer performance.

Performance optimization on the MIC platform, on the other hand, is a more involved activity and requires insight into compiler optimization techniques, cache behaviour, access patterns etc. However, these concepts are inherent principles of parallel programming even on multi-core processors and can be extended to MIC optimization seamlessly.

VI. CONCLUSIONS

In this era of power-aware computing, Intel's Many Integrated Core (MIC) co-processor is a new addition to the family of accelerators. This paper presents our experiences with the Intel MIC platform both from a performance and a programmability perspective. Using a well known and widely used signal and image processing algorithm - complex finite impulse response (FIR) filtering, we evaluate the ease of

programming on the MIC platform as well as its performance efficiency. Our evaluations indicate that Intel MIC holds a lot of promise to be an 'easy-to-program' accelerator of the future that also demonstrates good potential in terms of performance and scalability.

REFERENCES

- [1] M. Herlihy, "The multicore revolution: the challenges for theory," in *Proceedings of the 27th International Conference on Foundations of Software Technology and Theoretical Computer Science*. Berlin, Heidelberg: Springer-Verlag, 2007, pp. 1–8.
- [2] J. Shalf, "The new landscape of parallel computer architecture," *Journal of Physics: Conference Series*, vol. 78, no. 1, p. 012066, 2007.
- [3] A. R. Brodtkorb, C. Dyken, T. R. Hagen, J. M. Hjelmervik, and O. O. Storaasli, "State-of-the-art in heterogeneous computing," *Sci. Program.*, vol. 18, pp. 1–33, January 2010.
- [4] "Nvidia cuda c programming guide," http://developer.download.nvidia.com/compute/cuda/32/toolkit/docs/CUDA_C_Programming_Guide.pdf.
- [5] "Opencl - the open standard for parallel programming of heterogeneous systems," <http://www.khronos.org/opencl/>.
- [6] "Directcompute programming guide," http://developer.download.nvidia.com/compute/DevZone/docs/html/DirectCompute/doc/DirectCompute_Programming_Guide.pdf.
- [7] "Investigations of intel's many integrated core architecture at lrz," http://inside.hlr.de/htm/Edition_02_10/article_14.html.
- [8] "Cuda 4.0 release aims to make parallel programming easier," <http://gpgpu.org/2011/03/01/cuda-4-0-release>.
- [9] "Fir filter algorithm implementation using intel sse instructions," <http://download.intel.com/design/intarch/papers/323411.pdf>.
- [10] "The openmp api specification for parallel programming," <http://openmp.org/wp/>.
- [11] "Intel sse4 programming reference," <http://software.intel.com>.
- [12] "Intel advanced vector extensions programming reference," <http://software.intel.com>.
- [13] S. S. Muchnick, *Advanced compiler design and implementation*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1997.
- [14] K. Kennedy and J. R. Allen, *Optimizing compilers for modern architectures: a dependence-based approach*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2002.
- [15] D. F. Bacon, S. L. Graham, and O. J. Sharp, "Compiler transformations for high-performance computing," *ACM Comput. Surv.*, vol. 26, no. 4, pp. 345–420, Dec. 1994.
- [16] L. Seiler, D. Carmean, and E. S. et al, "Larrabee: a many-core x86 architecture for visual computing," in *Proceedings of SIGGRAPH vol 27 no 3*, November 2008.